

[Enunciado]

Se pide hacer una aplicación que permita jugar a una versión del juego **Merge Number Master** que está disponible tanto en plataformas Android como iOS:

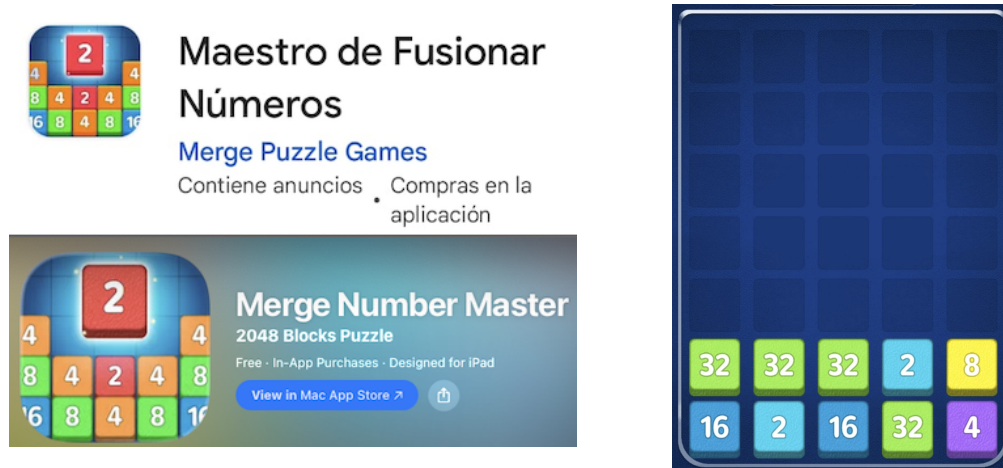


Figura 1: Logo de Merge Number Master y tablero al inicio de una partida en el juego original

Los objetivos primordiales del Trabajo en Grupo son dos:

1. Que practiquéis haciendo un programa que puede ser divertido/desafiante.
2. Que trabajéis y aprendáis con los compañeros de vuestro grupo.

Es decir, el objetivo no es que todos logréis hacer al final que el juego funcione perfectamente (aunque ojalá lo consigáis), sino que **el objetivo es que practiquéis**. Para programar bien hace falta practicar mucho y más ahora que estáis empezando. Aunque al final el juego no funcione, si habéis practicado haciéndolo, y habéis aprendido cosas, la práctica en grupo habrá servido para: 1) mejorar vuestro nivel como programadores, y 2) estar mejor preparados para el examen de enero. Ese es el motivo por el que la fecha de entrega coincide justo con el examen de enero, para que el Trabajo en Grupo os sirva para estudiar la asignatura.

Os recomendamos que el programa lo hagáis de una forma incremental. Empezando por las cosas sencillas poco a poco, probándolas bien hasta que estéis seguros de que funcionan, antes de pasar a las partes más complicadas. Aunque al final no funcione el juego, esas partes que estén bien serán tenidas muy en cuenta en la nota. Hacer muchas partes del programa de golpe y luego tratar de probarlas todas juntas complica mucho la depuración de errores. Es inevitablemente que tengáis errores en las versiones iniciales de cualquier parte del programa ya que muchas no son triviales y es normal cometer algunos errores al programar cada funcionalidad (incluso les ocurre a programadores con experiencia).

[Descripción general del juego]

Inicio del juego: El tablero, de un tamaño fijo 7 filas por 5 columnas, se inicializa con las 2 filas inferiores rellenas con números colocados al azar (ver Figura 1), estando el resto del tablero vacío. Los números con los que se rellenan las 2 filas inferiores son potencias de 2, desde el 2 hasta el 32. Además, se le pregunta al jugador cuál es el objetivo a alcanzar, bien a través de la consola o mediante un cuadro de diálogo (ver Figura 2). El objetivo tiene que ser una potencia de 2, comprendida entre 2 y 512, o una potencia de 2 en miles, es decir, 1K, 2K, 4K,...

Introduce tu objetivo: 2000

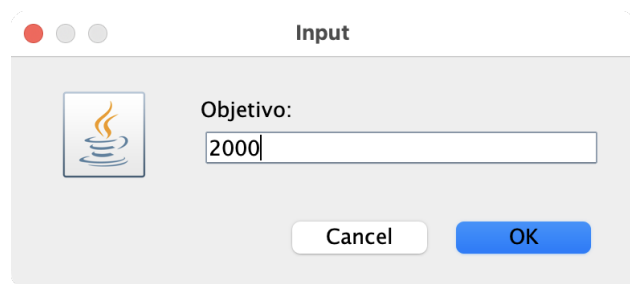


Figura 2: Petición del objetivo al inicio del juego, por la consola usando la clase Scanner o con un cuadro de diálogo usando JOptionPane.showInputDialog (ver Sesión 2 de prácticas)

Objetivo del juego: Llegar a conseguir una pieza del valor objetivo seleccionado. Para lograrlo, la jugada más habitual (no la única) consiste en fusionar 2 piezas iguales: la pieza origen se desplaza a donde está la pieza destino, y esta dobla su valor.

Jugadas: Las jugadas constan de los pasos siguientes:

1. **Elegir la pieza inicial que se va a mover:** El jugador tiene que elegir, indicando sus coordenadas, una pieza que se pueda mover hasta la casilla destino. Salvo que la pieza inicial y la casilla destino estén adyacentes en horizontal o vertical, eso implica que, en otros casos, para que la pieza inicial se pueda mover no tiene que tener piezas por arriba o por la derecha o por la izquierda. Si una pieza tiene números en las tres direcciones (arriba, a la izquierda y la derecha) entonces solo se podrá mover sobre otra casilla adyacente, pero la adyacencia es horizontal o vertical, no en diagonal. En la Figura 3 se muestra un ejemplo de las casillas movibles: son las que no tienen piezas arriba, a la izquierda o a la derecha (aparecen marcadas en azul), o las que tienen un casilla adyacente de su mismo número (los dos 32 en la fila inferior marcados en violeta). Nótese que el 8 que está en el centro de la fila inferior no puede moverse porque el otro 8 es adyacente pero en diagonal.

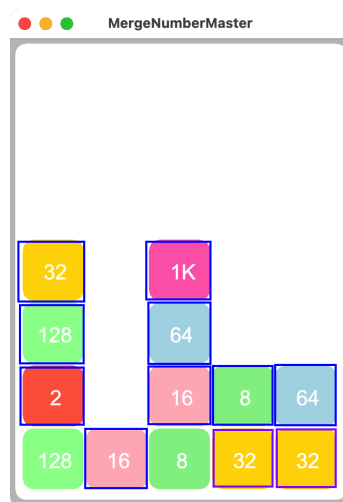


Figura 3: Posibles piezas iniciales, piezas movibles (recuadradas en azul) o que tienen una pieza adyacente en horizontal o vertical (recuadradas en violeta)

2. **Elegir la casilla destino:** El jugador tiene que elegir, indicando sus coordenadas, la casilla destino que puede ser: 1) Una pieza que esté justo adyacente (por arriba, por la izquierda o por la derecha, pero no en diagonal) a la pieza inicial. 2) Una pieza que también se pueda mover. 3) La casilla libre que está justo en el tope de cada columna. La idea es que la pieza inicial se “desplaza” por el tablero sin chocar con otra pieza hasta llegar a la casilla destino, por eso ambas casillas deben ser “accesibles” la una para la otra, no pueden estar rodeadas por otras piezas que impidan el desplazamiento de la pieza inicial hasta la casilla destino.

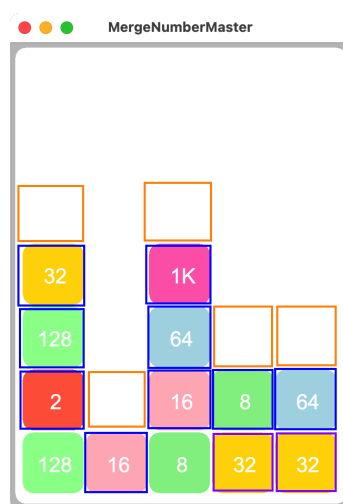


Figura 4: Posibles casillas destino, son las mismas que las piezas iniciales de la Figura 3, y además el tope de cada columna (cuadrados en naranja)

3. **Comprobar que la jugada es válida:** Hay que comprobar que la jugada permite fusionar 2 piezas iguales de alguna forma. Hay varias formas como describiremos más adelante. Si es así, la jugada es válida. Si la jugada no es válida, todas las piezas del tablero se desplazarán hacia arriba una fila y se generará una fila con piezas aleatorias en la parte inferior.
4. **Realizar la jugada:** Si la jugada es válida se hace el desplazamiento indicado de la pieza inicial a la casilla destino y se fusionan las piezas que correspondan.
5. **Añadir una fila en parte inferior del tablero:** Si se ha elegido una jugada que no provoca una fusión o si es imposible hacer una jugada que provoque una fusión, entonces se genera una nueva fila en la parte inferior de forma aleatoria.
6. **Realizar fusiones:** Además de hacer la fusión producto de una jugada, es posible que se generen fusiones nuevas con la pieza generada tras la fusión de una jugada, o cuando se añade una fila por abajo.

Fin de la partida: El juego se acaba cuando el jugador alcanza la cifra objetivo seleccionada al inicio de la partida (victoria) o cuando al menos una de las columnas del tablero está llena y se tiene que añadir una fila en la parte inferior tras hacer una jugada incorrecta o porque no se pueda hacer una jugada válida (derrota). El programa mostrará un mensaje indicando el final de la partida y si el jugador ha ganado o perdido la partida.

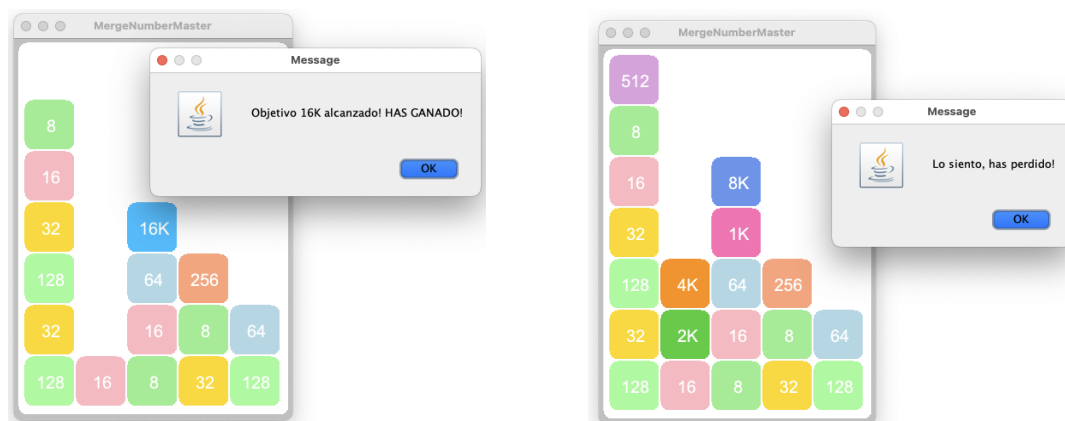


Figura 5: Fin de partida: por victoria (objetivo alcanzado, figura izquierda) o derrota, en el tablero no se puede hacer ninguna jugada válida y la primera columna está llena (figura derecha)

En los siguientes párrafos describimos con ejemplos de forma más precisa alguna de estas partes, en especial las jugadas y algunos detalles que hay que tener en cuenta para implementar otras partes del juego.

[Tablero]

Lo primero que debe hacer el programa es crear el Tablero y rellenar al azar las 2 filas inferiores con números entre el 2, 4, 8, 16 y 32. Para ello os recomendamos emplear la clase `Random` que usamos en alguna de las prácticas de laboratorio. También hay que mostrar de alguna forma el tablero al jugador.

[Jugadas válidas]

Vamos a ver los 4 tipos de jugadas que generan una jugada válida, de menor a mayor grado de dificultad (os recomendamos empezar por las jugadas más sencillas):

1. Que la pieza inicial que se va a mover y la casilla destino estén adyacentes, teniendo la casilla destino una pieza igual a la pieza que se ha decidido mover. La adyacencia es en horizontal o vertical, nunca en diagonal. Realmente solo hay que comprobar a la izquierda, a la derecha o arriba. No hace falta mirar abajo, salvo que no hagáis la parte de fusiones. Si se hacen todos los tipos de fusiones, nunca debería haber una pieza encima de otra de igual valor. Ver Figura 6.

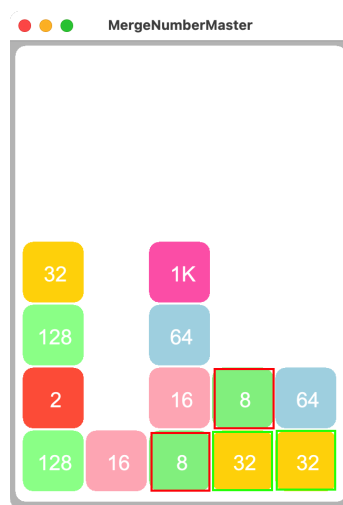


Figura 6: Piezas adyacentes. Las piezas con el valor 32 (recuadradas en verde) son adyacentes. Sin embargo, las piezas con el valor 8 (recuadradas en rojo) no lo son, porque están en diagonal

2. Que tanto la pieza inicial como la casilla destino sea “movibles” y que la casilla destino tenga una pieza igual que la pieza que se ha decidido mover. El que sean movibles significa que no están rodeadas por piezas en todas las direcciones, es decir, que hay alguna dirección (arriba, a la izquierda o la derecha) en la que hay una casilla vacía. Ver Figura 7.

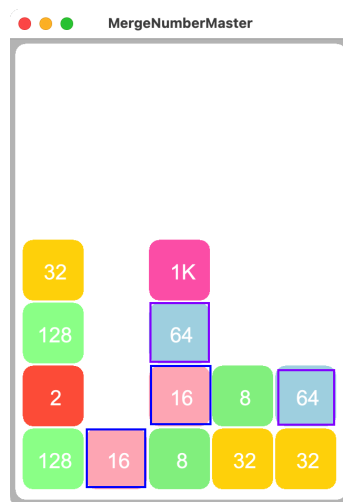


Figura 7: Piezas iguales. Las piezas con el valor 16 (recuadradas en azul) y las piezas con el valor 64 (en violeta) se pueden mover y provocar una fusión válida

3. Que la pieza inicial sea movable y que la casilla destino sea la casilla en blanco encima de una columna. Se tiene que cumplir además, que la casilla justo debajo de la casilla destino tenga una pieza igual que la pieza que se ha decidido mover. En ese caso, esas dos piezas se fusionan al “caer” la pieza que se ha movido sobre la otra. Ver Figura 8.

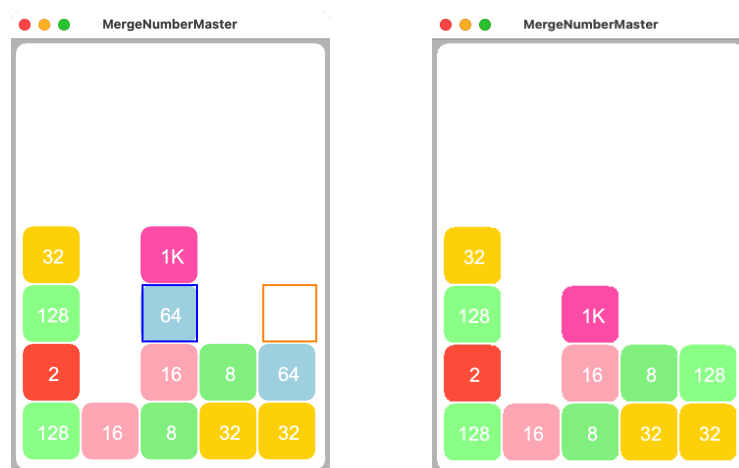


Figura 8: Jugada con casilla destino el tope de una columna. La pieza 64 (recuadrada en azul) se puede mover a la casilla recuadrada en naranja y provocar una fusión válida “al caer” sobre la pieza que tienen debajo, también un 64

4. Que la pieza inicial sea movable y que las piezas que tiene por encima y por debajo sean del mismo valor. En ese caso, aunque la propia pieza no genere una fusión, que podría producirla también, esta se produce al quitar esa

pieza. Ver Figura 9. Recaltar que **este tipo de fusiones se pueden producir a la vez que las fusiones producidas por los otros tipos de jugadas.**

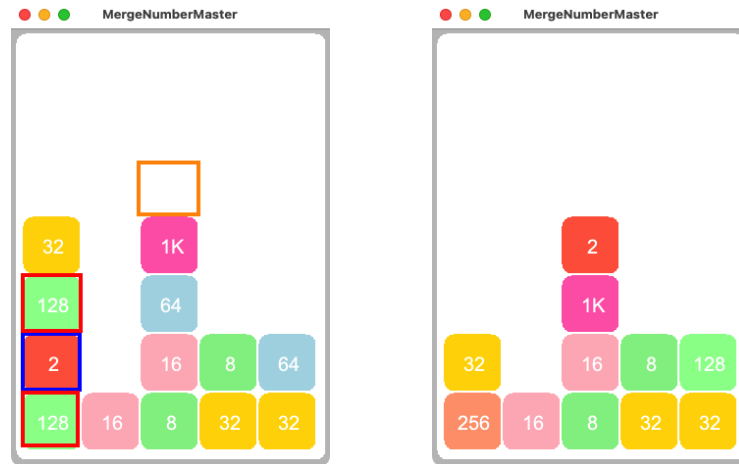


Figura 9: Fusión por eliminación. La pieza 2 (recuadrada en azul) se mueve a la casilla recuadrada en naranja y provoca que las piezas con valor 128 que tiene por encima y debajo (recuadradas en rojo) se fusionen generando el 256

[Añadir fila abajo]

Hay que añadir una fila abajo en dos situaciones. 1) Cuando se hace una jugada y no se produce ninguna fusión. Este caso es fácil de detectar. 2) Cuando realmente es imposible hacer una jugada válida. Este caso es más complicado. Requiere que, antes de que el jugador elija la nueva jugada, se haga un proceso de búsqueda para ver si es posible hacer alguna jugada o no, ver por ejemplo la Figura 10.

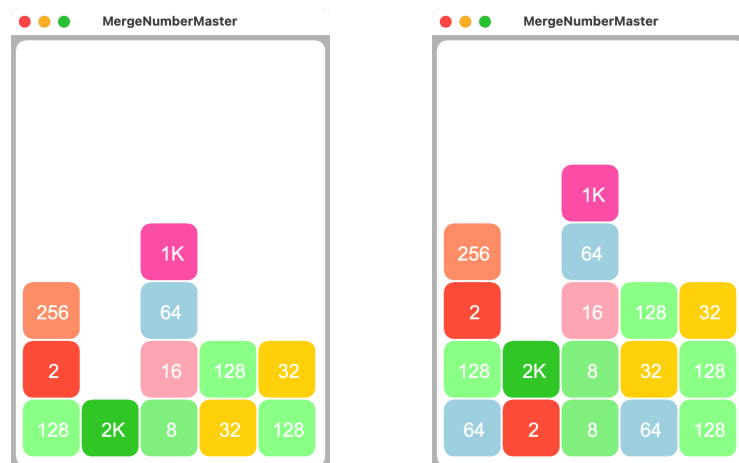


Figura 10: No hay jugadas posibles. En el tablero de ejemplo no hay jugadas posibles, se tiene que añadir una nueva fila

Añadir una nueva fila implica 3 tareas: 1) Desplazar todas las casillas del tablero una fila hacia arriba. 2) Añadir los números aleatorios en la última fila. Para añadir un poco de dificultad a esta parte, **la nueva fila generada tiene que tener valores aleatorios que sean estrictamente menores que la pieza más alta que haya en el tablero**. Esto lo hacemos para no ayudar demasiado al jugador a alcanzar el objetivo. De otro modo, se podría generar una pieza igual o mayor que el objetivo. 3) Hacer las fusiones automáticas que surjan (ver siguiente sección).

[Fusionar piezas]

Las fusiones automáticas, es decir, las que no son producto de una jugada directamente, sino que se producen tras una jugada, o al añadir una fila, es la parte más difícil del juego, mucho más que las jugadas en sí. Lo primero que hay que tener claro es que **las fusiones automáticas solo se hacen en vertical**, nunca en horizontal. Es decir, si hay dos piezas iguales en horizontal, nunca hay que hacer una fusión. Esto lo simplifica, pero no hace que sea sencillo. Vamos a analizar las dos situaciones donde se pueden generar fusiones:

1. Que al quitar la pieza inicial se produzcan fusiones en esa columna, y/o que se produzcan fusiones en la columna de la casilla destino después de hacer la fusión de la jugada. Ver el ejemplo de la Figura 11. Si elegimos el 8 de la primera columna y lo movemos al 8 ocho de la última columna, se producirán fusiones en ambas columnas. En la primera columna se fusionarán primero los dos 32, se generará 64, que ese fusionará con el 64 de abajo. Y en la última columna se producirán 3 fusiones sucesivas: $16+16=32$, $32+32=64$, $64+64=128$.

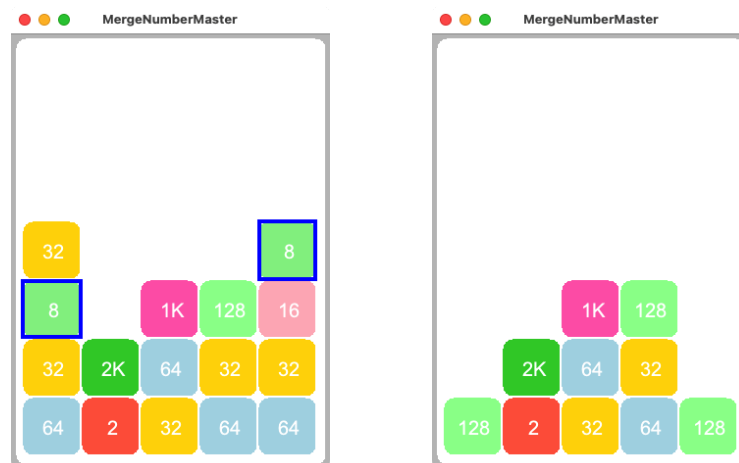


Figura 11: Fusiones al hacer una jugada. La pieza 8 de la primera columna se mueve sobre el 8 de la última columna. En ambas columnas ocurren varias fusiones automáticas, produciendo al final una sola pieza de valor 128 en cada columna

El caso descrito es complejo, pero queda claro que **tras hacer una jugada**

que produzca una fusión válida, hay que comprobar si se tienen que hacer fusiones automáticas en la columna de la pieza inicial y en la columna de la casilla destino.

2. Que al añadir una nueva fila nueva se generen fusiones con la pieza nueva que se ha insertado en cada columna de la última fila. Este caso es más sencillo que el anterior, ya que la fusión siempre se produce sobre esa pieza que está en la última fila. Puede que haya más de una fusión. Estas fusiones hay que hacerlas columna a columna. Por ejemplo, en el caso de la Figura 12 el 8 que está en la columna del centro primero se fusionará automáticamente con el 8 que tiene encima, generando una pieza de valor 16. Las piezas de encima caen hacia abajo, y tendremos de nuevo otra fusión ya que hay 2 piezas de valor 16, una encima de la otra, generando una pieza de valor 32. En cambio, el 128 de la última fila, última columna, solo se fusiona una vez con el 128 que tiene justo encima, genera una pieza de valor 256, y simplemente las piezas que tiene encima, en este caso, el 32, cae, pero ya no se generan nuevas fusiones. Puede ocurrir que, tras hacer las fusiones que se producen al añadir una nueva fila, no haya jugadas válidas. En ese caso se tendría que añadir una nueva fila y repetir el proceso de comprobar fusiones automáticas.

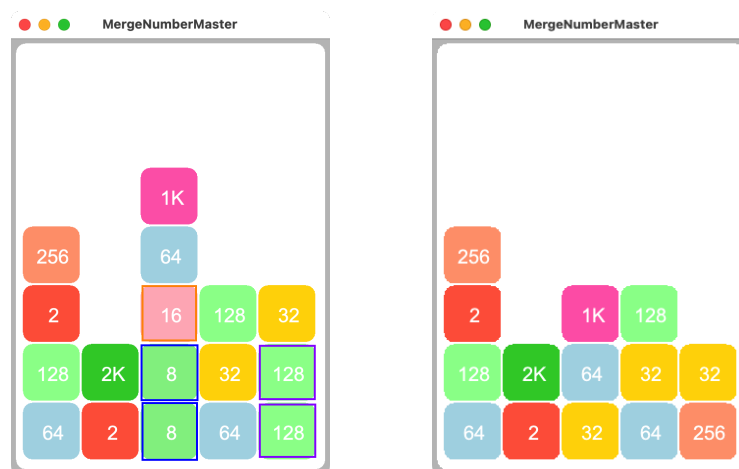


Figura 12: Fusiones al añadir la última fila. La pieza 8, encuadrada en azul, generará 2 fusiones, primero con el 8 que tiene encima, generando un 16, y luego con el 16 (encuadrado en naranja) al caer sobre él. En cambio, la pieza 128, encuadrada en violeta, solo genera una fusión

[Opciones para realizar la práctica]

Hay dos opciones para realizar la práctica:

1. **Aplicación de consola:** el tablero se muestra en formato texto en la pantalla y se leen las jugadas de teclado.
2. **Aplicación gráfica:** se hace una aplicación gráfica similar a la vista para ejemplificar las jugadas en las páginas anteriores.

[Aplicación de consola]

Se jugará a través de la consola, mostrando el estado del tablero en la pantalla y leyendo los datos de las casillas elegidas a través del teclado. Se recomienda mostrar el estado del tablero tras cada uno de los pasos de las jugadas para comprobar que el juego está funcionando correctamente. En la figura siguiente aparece un ejemplo de funcionamiento de la aplicación de consola:

```
Introduce tu objetivo: 2000
      1      2      3      4      5
-----
1 |
2 |
3 |
4 |
5 |
6 | 32      4      32      16      4
7 | 4      16      4      8      8

Intro fila y columna de la pieza a mover y fila y columna de su destino: 7 4 7 5
      1      2      3      4      5
-----
1 |
2 |
3 |
4 |
5 |
6 | 32      4      32      16      4
7 | 4      16      4      16      16

Bajamos piezas
      1      2      3      4      5
-----
1 |
2 |
3 |
4 |
5 |
6 | 32      4      32      16      4
7 | 4      16      4      16      16

Intro fila y columna de la pieza a mover y fila y columna de su destino: 6 1 6 3
      1      2      3      4      5
-----
1 |
2 |
3 |
4 |
5 |
6 |      4      16      64      4
7 | 4      16      4      16      16

Intro fila y columna de la pieza a mover y fila y columna de su destino: 7 1 6 2
      1      2      3      4      5
-----
1 |
2 |
3 |
4 |
5 |
6 |      8      64      4
7 |      16      4      16      16
```

Figura 13: Funcionamiento de la aplicación de consola

Calificación si se hace mediante una aplicación de consola:

- La nota máxima de la aplicación de consola será de 8.5 puntos si se implementa la versión del juego con todas las funcionalidades descritas, incluyendo las más difíciles que son: 1) detectar que no hay jugadas posibles y añadir una nueva fila, 2) hacer todos los tipos de fusiones posibles, incluyendo las que implican fusiones recurrentes.

[Aplicación gráfica]

Se jugará a través de una ventana gráfica mostrando el estado del tablero mediante una representación similar a la que aparece en las figuras que describen el juego al inicio de este documento.

Las jugadas se realizarán mediante clicks de ratón, pinchando el jugador en la pieza inicial elegida y luego en la casilla destino.

Aunque la aplicación sea gráfica, se debe mostrar por la consola el estado del tablero tras cada paso de las jugadas de forma que se pueda comprobar que el juego funciona correctamente. Hacer que la visualización del tablero se detenga tras cada uno de esos momentos NO es trivial y **NO se pide**. Si algún grupo desea hacerlo puede solicitar un tutoría.

Calificación si se hace mediante una aplicación gráfica:

- Si el juego se implementa en modo gráfico, se pueden alcanzar los 10 puntos incluso aunque haya alguna cosa incorrecta en función de su importancia. Es decir, si se hace en modo gráfico, pero a lo mejor no funcionan todas las fusiones, o no se ha hecho la funcionalidad para detectar que no hay una jugada posible, se pueden alcanzar los 10 puntos. La idea es premiar a aquellos alumnos que más cosas intenten y logren hacer.

[Normas]

1. El programa debe ser original y realizado íntegramente por los componentes del grupo. Si se detecta cualquier tipo de copia, de cualquier fuente o procedencia, los integrantes del grupo o los grupos que se vean implicados **suspenderán la asignatura**. Suspenden también los alumnos que pasan la práctica, no solamente los que la copian.
2. Se debe seguir el paradigma de la Programación orientada a objetos, es decir, se debe escribir una clase que implemente el juego y un programa que emplee dicha clase para realizar la aplicación.
3. Cada grupo, formado por un máximo de 3 alumnos, debe entregar en un documento zip lo siguiente:
 - Los ficheros .java que componen la aplicación.
 - Una memoria en formato pdf de no más de 4 páginas en el que se expliquen los siguientes aspectos:
 - a) La descripción de las clases empleadas.
 - b) La forma de representar el tablero y de colocar las fichas.
 - c) Los esquemas iterativos empleados para realizar el juego, en particular los que tienen que ver con la realización de las jugadas.
 - d) **(Muy Importante)** Una lista con todos aquellos elementos que NO funcionen en la aplicación.
 - e) **NO se deben incluir listados de la aplicación en la memoria.**

[Se valorarán los siguientes aspectos]

- Que se empleen las metodologías de programación que se han estudiado durante el curso, p.e, en cuanto al diseño de clases y de esquemas iterativos.
- La eficiencia del programa.
- Los comentarios incluidos en el programa, Javadoc y normales.
- La claridad y sencillez del código.
- **Defensa:** los alumnos de un grupo pueden tener que defender su práctica antes los profesores. Todos los miembros del grupo deben entender cómo están hechas todas las partes del programa aunque hayan sido hechas por otro miembro del grupo. Las defensas se realizarán los días 27 y 28 de enero.

Hora y fecha límite de entrega: las 08:59 horas del día 14 de enero de 2026. La entrega se debe hacer a través del Campus Virtual en la Tarea que se habilitará al efecto (basta con que la suba un alumno del grupo).

[Plantilla para realizar la aplicación gráfica]

Lo que sigue a continuación es el código fuente de las dos clases que integran el proyecto `Tablero` que se suministra con este enunciado: la clase `Tablero` que puede servir de base para la implementación del juego, y la clase `TestTablero` que constituiría el programa principal. Si se ejecuta tal como se suministra, el programa captura los click del ratón e indica en un cuadro de diálogo la posición (en pixeles) donde se produjo el click.

En el fichero `Tablero.java` se encuentra implementado mediante la clase privada `MouseHandler` la manera de detectar los clicks de ratón y las coordenadas donde se producen.

Cómo mostrar un cuadro de diálogo (`JOptionPane.showMessageDialog()`) se estudió en las sesiones prácticas de la asignatura. Para dibujar cuadrados se puede utilizar el método `fillRect()`, para dibujar líneas puede emplearse el método `drawLine()` y para escribir textos `drawString()`. Todos esos métodos con un objeto de la clase `Graphics` como se muestra en el siguiente ejemplo.

TestTablero.java

```
1 import javax.swing.JFrame;
2 import javax.swing.JOptionPane;

4 public class TestTablero {

6     public static void main(String[] args) {
7         Tablero t = new Tablero();

9         JFrame app = new JFrame("Tablero");

11        String entrada= JOptionPane.showInputDialog("Filas:");
12        int filas =Integer.parseInt(entrada);

14        String salida=String.format("Has elegido %d filas", filas );
15        JOptionPane.showMessageDialog(null, salida);

17        app.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
18        app.setBounds(100, 100, 440, 470);
19        app.add(t);
20        app.setVisible(true);
21    }
22 }
```

Tablero.java

```
1 import java.awt.Color;
2 import java.awt.Font;
3 import java.awt.Graphics;
4 import java.awt.event.MouseAdapter;
5 import java.awt.event.MouseEvent;
6 import javax.swing.JOptionPane;
7 import javax.swing.JPanel;

10 public class Tablero extends JPanel {

12     // Aquí irían los atributos necesarios

14     // Constructores
15     Tablero() {
16         // El constructor debe tener los parámetros oportunos
17         // para inicializar el tablero y el juego

19         // Añadimos el 'escuchador' de ratón
20         addMouseListener(new MouseHandler());
21     }

23     // Métodos de la clase que implementan el juego: básicamente hacer una
24     // jugada, dibujar el estado del tablero y comprobar si la partida se acabó

26     // Método paint
27     public void paintComponent(Graphics g) {
28         super.paintComponent(g);

30         // Aquí iría el código para pintar el estado del tablero

32         // Pintar rectángulo
33         g.setColor(Color.blue);
34         g.fillRect(10, 10, 40, 40);

36         // Pintar línea
37         g.setColor(Color.lightGray);
38         g.drawLine(55, 10, 55, 80);

40         // Pintar un texto
41         Font fuente = new Font("Arial", Font.PLAIN, 20);
42         g.setFont(fuente);
43         g.setColor(Color.black);
```



```
44     g.drawString("Texto", 10,100);
45 }

47 //Clase privada para capturar los eventos del ratón
48 private class MouseHandler extends MouseAdapter {
49     public void mouseClicked (MouseEvent e) {
50         //Mostramos un diálogo con la posición del ratón
51         //para ver un ejemplo de cómo se obtienen las coordenadas
52         //donde se produjo el click
53         JOptionPane.showMessageDialog(null, String.format("Ratón %d
                    ,%d \n",e.getX(),e.getY()));

55         //Aquí irían las instrucciones para comprobar si el
56         //click del ratón se produjo en una posición correcta
57         //y hacer la jugada correspondiente

59         //Se pueden llamar a los métodos públicos de la clase

61         //Seguramente habrá que repintar el tablero si se realizó
62         //una jugada válida
63         repaint() ;
64     }
65 }
66 }
```
